Лабораторная работа №3
«Устройства хранения данных»

1. Цель работы

Изучить архитектуру и протоколы современных устройств хранения данных (HDD, SSD, NVMe); освоить программные интерфейсы операционной системы для обнаружения накопителей, получения их идентификационной информации, параметров производительности и состояния (SMART); приобрести практические навыки разработки приложений на C++ с использованием низкоуровневых системных API (Win32 API, Linux sysfs, ioctl, прямой доступ к портам ввода-вывода) без применения высокоуровневых библиотек и обёрток (WMI, .NET, libatasmart); научиться анализировать состояние накопителей в режиме реального времени и формировать диагностические отчёты.

2. Задачи

1. Изучить эволюцию интерфейсов хранения данных от параллельного ATA до современных SATA и NVMe.
2. Изучить структуру ATA-регистров и протоколы обмена данными (PIO, DMA, UDMA) как теоретическую базу.
3. Освоить механизмы обнаружения физических и логических накопителей через системный API (SetupAPI / sysfs / udev).
4. Реализовать получение и отображение основных сведений об устройстве: наименование, тип (HDD/SSD), объём, версия прошивки, интерфейс подключения.
5. Реализовать получение параметров производительности: скорость чтения/записи интерфейса, время отклика, загрузка диска.
6. Реализовать получение информации о состоянии накопителя: температура, количество часов работы, циклы включения, оценка здоровья (Health Status) через SMART.
7. Реализовать анализ использования пространства: свободная/занятая память, процент заполнения.
8. Обеспечить отслеживание изменений состояния накопителя в режиме реального времени.
9. Реализовать ведение журнала событий и формирование отчёта о текущем состоянии устройства.
10. Изучить механизмы доступа к накопителям в Linux: sysfs, ioctl (HDIO_GET_IDENTITY, SG_IO, NVMe ioctl), /proc/diskstats, прямые порты ATA.
11. Разработать графический пользовательский интерфейс приложения в соответствии с установленным в ЛР №1 стилем оформления.

Интерфейсы и устройства вычислительных машин

3. Теоретический материал

3.1. Эволюция интерфейсов хранения данных

ATA (AT Attachment) — интерфейс, разработанный в 1986–1990 гг. для подключения накопителей на жёстких магнитных дисках к компьютерам IBM PC AT с шиной ISA. Стандарт определяет набор регистров устройства и назначение сигналов 40-контактного интерфейсного разъёма. Название IDE (Integrated Drive Electronics) стало неофициальным синонимом ATA.

Стандартный контроллер ATA позволял подключать до двух накопителей на один канал: **ведущее устройство (Master / Device 0)** и **ведомое устройство (Slave / Device 1)**. Выбор устройства осуществлялся битом DEV в регистре DH.

SATA (Serial ATA) — последовательный интерфейс, пришедший на смену параллельному ATA (PATA) в начале 2000-х. SATA использует дифференциальную пару для передачи данных, что обеспечивает более высокие скорости (1.5, 3, 6, 12 Гбит/с), горячее подключение и упрощённую кабельную разводку. Для работы с SATA-накопителями в современных системах используется контроллер **AHCI (Advanced Host Controller Interface)**.

NVMe (Non-Volatile Memory Express) — протокол и логический интерфейс для доступа к твердотельным накопителям (SSD), подключённым через шину PCI Express. NVMe был разработан специально для NAND-памяти и обеспечивает значительно более низкую латентность и более высокую пропускную способность по сравнению с SATA/AHCI. NVMe-накопители используют форм-факторы M.2, U.2, PCIe Add-in Card.

3.2. Архитектура ATA/ATAPI

Интерфейс ATA определяет следующие компоненты:

- **хост-адаптер** — средства сопряжения интерфейса ATA с шиной компьютера (хост-контроллер);
- **ведущее устройство (Master / Device 0)** — жёсткий диск с интегрированным контроллером, подключённый к хост-контроллеру;
- **ведомое устройство (Slave / Device 1)** — второй накопитель на том же канале.

Для выбора устройства, исполняющего текущую команду, используется бит выбора устройства (DEV) в регистре номера устройства и головки (DH). Если бит DEV = 0, выбрано ведущее устройство; если DEV = 1 — ведомое. Запись в этот регистр воспринимается сразу обоими устройствами, но на обращения к остальным регистрам реагирует только выбранное.

3.3. Регистры устройства ATA

Каждое устройство ATA имеет стандартный набор регистров, адресуемых сигналами от хост-адаптера (CS0#, CS1#, DA00–DA02, DIOR# и DIOW#). Набор

Интерфейсы устройств вычислительных машин

регистров состоит из двух блоков, выбираемых сигналами CS0# и CS1#.

3.3.1. Управляющие регистры (CS0# = 1, CS1# = 0)

Адрес порта (канал 1)	Адрес порта (канал 2)	Чтение	Запись
0x3F6	0x376	AS — альтернативный статус	DC — управление устройством
0x3F7	0x377	DA — адрес устройства	—

Регистр управления устройством DC (адрес 3F6h/376h):

- бит 2 — **SRST** (Software Reset): программный сброс, действует всё время, пока бит не будет снят (оба устройства на шине воспринимают сброс одновременно);
- бит 1 — **nIEN** (Interrupt Enable): инверсный бит разрешения прерывания.

3.3.2. Командные регистры (CS0# = 0, CS1# = 1)

Адрес порта (канал 1)	Адрес порта (канал 2)	Чтение	Запись
0x1F0	0x170	DR — регистр данных	DR — регистр данных
0x1F1	0x171	ER — регистр ошибок	FR — регистр свойств
0x1F2	0x172	SC — счётчик секторов	SC — счётчик секторов
0x1F3	0x173	SN — номер сектора или LBA [7:0]	SN — номер сектора или LBA [7:0]
0x1F4	0x174	CL — младший байт номера цилиндра или LBA [15:8]	CL — младший байт номера цилиндра или LBA [15:8]
0x1F5	0x175	CH — старший байт номера цилиндра или LBA [23:16]	CH — старший байт номера цилиндра или LBA [23:16]
0x1F6	0x176	DH — номер устройства и головки или LBA [27:24]	DH — номер устройства и головки или LBA [27:24]
0x1F7	0x177	SR — регистр состояния	CR — регистр команд

Интерфейсы и устройства вычислительных машин

Назначение ключевых регистров:

- **DR (Data Register)** — используется для обмена данными в режиме PIO;
- **ER (Error Register)** — хранит состояние выполнения последней операции или диагностический код;
- **FR (Features Register)** — используется в команде Set Features для задания подкоманд;
- **SC (Sector Count)** — содержит число секторов, участвующих в обмене;
- **SN, CL, CH, DH** — регистры адреса (CHS или LBA);
- **SR (Status Register)** — отражает текущее состояние устройства;
- **CR (Command Register)** — запись кода команды инициирует её выполнение.

3.3.3. Регистр состояния SR (Status Register)

Бит	Название	Описание
7	BSY (Busy)	Устройство занято. При BSY = 1 команды игнорируются.
6	DRDY (Device Ready)	Устройство готово к восприятию команд.
5	DF (Device Fault)	Индикатор отказа устройства.
4	DSC (Device Seek Complete)	Завершение поиска трека.
3	DRQ (Data Request)	Готовность к обмену данными.
2	CORR (Corrected Data)	Исправленная ошибка данных.
1	IDX (Index)	Индекс (трактуется производителем).
0	ERR (Error)	Ошибка выполнения предыдущей операции.

3.4. Протоколы обмена данными

3.4.1. Программный доступ PIO (Programmed Input/Output)

Выполняется в виде последовательных операций чтения/записи в пространство ввода-вывода по адресу регистра данных. Готовность устройства проверяется перед началом передачи блока. Обмен PIO программно реализуется с помощью процессорных инструкций ввода-вывода строк REP INS / REP OUTS. Традиционные режимы 0, 1 и 2 имеют фиксированные временные

Интерфейсы и устройства вычислительных машин

параметры; прогрессивные режимы ATA-2 (PIO Mode 3 и выше) позволяют устройству затормозить обмен сигналом готовности IORDY.

3.4.2. Обмен по каналу DMA

Занимает исключительно шины ввода-вывода и памяти. Процессору требуется выполнить только процедуру инициализации канала DMA, после чего он свободен до прерывания от устройства в конце передачи блока.

- **Singleword DMA** — устройство для передачи каждого слова вырабатывает сигнал запроса DMARQ;
- **Multiword DMA** — хост отвечает потоком циклов, сопровождаемых сигналами DMACK#.

3.4.3. Ultra DMA (UDMA)

Протокол, позволяющий перешагнуть барьер в 16,6 Мбайт/с, свойственный традиционным режимам. Обеспечивается контроль достоверности передачи данных по шине через подсчёт CRC-кода. Стандартом ATA-4 определены три режима Ultra DMA (0, 1 и 2), впоследствии введены новые (до UDMA-6 / 133 Мбайт/с).

3.5. Современные интерфейсы: SATA, AHCI, NVMe

3.5.1. SATA и AHCI

SATA (Serial ATA) использует последовательную передачу данных. Ключевые особенности:

- скорости передачи: 1.5 (SATA I), 3.0 (SATA II), 6.0 (SATA III), 12 (SATA 3.2) Гбит/с;
- горячее подключение (Hot Plug);
- NCQ (Native Command Queuing) — встроенная очерёдность команд;
- **TRIM** — команда информирования SSD о неиспользуемых секторах.

AHCI (Advanced Host Controller Interface) — спецификация, определяющая работу хост-контроллера SATA. AHCI поддерживает NCQ, горячее подключение и энергосбережение.

3.5.2. NVMe

NVMe — протокол для доступа к энергонезависимой памяти через PCI Express:

- использует очереди команд (Submission Queues / Completion Queues) в системной памяти;
- поддерживает до 64K очередей с глубиной до 64K команд каждая;
- задержка (latency) значительно ниже, чем у SATA/AHCI;
- форм-факторы: M.2, U.2, PCIe AIC.

Интерфейсы устройства вычислительных машин

3.6. SMART — технология самодиагностики

SMART (Self-Monitoring, Analysis and Reporting Technology) — технология оценки состояния накопителя путём мониторинга различных атрибутов. Для ATA-накопителей SMART-данные получаются командой **IDENTIFY DEVICE** (0xEC) и командами чтения SMART-атрибутов.

Ключевые SMART-атрибуты (ID):

- **0x01** — Raw Read Error Rate (частота ошибок чтения);
- **0x05** — Reallocated Sectors Count (число переназначенных секторов);
- **0x09** — Power-On Hours (часы работы);
- **0x0C** — Power Cycle Count (число циклов включения/выключения);
- **0xC2** — Temperature (температура);
- **0xC7** — UltraDMA CRC Error Count (ошибки CRC интерфейса).

Для **NVMe** информация о здоровье содержится в **SMART/Health Information Log** (Log Page 0x02):

- CriticalWarning — критические предупреждения;

Temperature — температура контроллера (в Кельвинах);

- AvailableSpare — доступный резервный объём (%);
- PercentageUsed — процент износа (для SSD);
- DataUnitsRead / DataUnitsWritten — объём прочитанных/записанных данных;
- PowerCycles / PowerOnHours — циклы включения и часы работы.

3.7. Win32 API для работы с накопителями (Windows)

3.7.1. Обнаружение накопителей

Перечисление физических дисков через SetupAPI:

```
#include <setupapi.h>
#include <initguid.h>
#include <ntddstor.h>

HDEVINFO hDevInfo = SetupDiGetClassDevs(
    &GUID_DEVINTERFACE_DISK, NULL, NULL,
    DIGCF_PRESENT | DIGCF_DEVICEINTERFACE
);
// Перебор устройств через SetupDiEnumDeviceInterfaces +
SetupDiGetDeviceInterfaceDetail
```

Перечисление логических томов:

```
DWORD drives = GetLogicalDrives(); // Битовая маска дисков A-Z
UINT type = GetDriveType(L"C:\\"); // DRIVE_FIXED, DRIVE_REMOVABLE и
т.д.
```

Получение информации о свободном месте:

Интерфейсы устройства вычислительных машин

```
ULARGE_INTEGER freeBytes, totalBytes, totalFreeBytes;
GetDiskFreeSpaceEx(L"C:\\", &freeBytes, &totalBytes,
&totalFreeBytes);
```

3.7.2. Открытие физического диска

```
HANDLE hDevice = CreateFile(
    L"\\\\.\\PhysicalDrive0",        // Имя устройства
    GENERIC_READ | GENERIC_WRITE,   // Доступ
    FILE_SHARE_READ | FILE_SHARE_WRITE,
    NULL, OPEN_EXISTING, 0, NULL
);
```

3.7.3. Получение свойств устройства (IOCTL_STORAGE_QUERY_PROPERTY)

```
STORAGE_PROPERTY_QUERY query = {0};
query.PropertyId = StorageDeviceProperty;
query.QueryType = PropertyStandardQuery;

STORAGE_DEVICE_DESCRIPTOR desc = {0};
DWORD bytesReturned = 0;

DeviceIoControl(hDevice, IOCTL_STORAGE_QUERY_PROPERTY,
    &query, sizeof(query),
    &desc, sizeof(desc),
    &bytesReturned, NULL);
```

Возвращаемая информация:

- VendorId, ProductId, ProductRevision — производитель, модель, версия прошивки;
- SerialNumber — серийный номер;
- BusType — тип шины (BusTypeSata, BusTypeNvme, BusTypeUsb и т.д.).

Для определения типа накопителя (HDD/SSD):

```
// StorageDeviceSeekPenaltyProperty — наличие штрафа на
позиционирование
// StorageDeviceTrimProperty — поддержка TRIM (характерно для SSD)
// StorageDeviceRPMProperty — число оборотов (1 для SSD, 0 —
неизвестно, >1 для HDD)
```

3.7.4. Получение геометрии и размера диска

```
DISK_GEOMETRY_EX geomEx;
DeviceIoControl(hDevice, IOCTL_DISK_GET_DRIVE_GEOMETRY_EX,
    NULL, 0, &geomEx, sizeof(geomEx), &bytesReturned, NULL);
// geomEx.DiskSize — полный размер диска в байтах
// geomEx.Geometry.BytesPerSector, SectorsPerTrack,
TracksPerCylinder
```

Интерфейсы и устройства вычислительных машин

3.7.5. SMART для SATA-накопителей (IOCTL_ATA_PASS_THROUGH)

```
ATA_PASS_THROUGH_EX apt = {0};
apt.Length = sizeof(ATA_PASS_THROUGH_EX);
apt.AtaFlags = ATA_FLAGS_DATA_IN;
apt.DataTransferLength = 512;
apt.TimeOutValue = 5;
apt.DataBufferOffset = offsetof(ATA_PASS_THROUGH_EX, DataBuffer);

// Команда IDENTIFY DEVICE (0xEC)
apt.CurrentTaskFile[6] = 0xEC;

BYTE buffer[sizeof(ATA_PASS_THROUGH_EX) + 512] = {0};
memcpy(buffer, &apt, sizeof(apt));

DeviceIoControl(hDevice, IOCTL_ATA_PASS_THROUGH,
    buffer, sizeof(buffer), buffer, sizeof(buffer), &bytesReturned,
NULL);

// Данные IDENTIFY DEVICE находятся после заголовка
ATA_PASS_THROUGH_EX
// Word 217 — Nominal Media Rotation Rate (0x0001 = SSD)
```

Для чтения SMART-атрибутов используется команда **SMART READ DATA** (0xB0 с подкомандой 0xD0).

3.7.6. SMART/Health для NVMe-накопителей

```
// Запрос NVMe Health Information Log через
IOCTL_STORAGE_QUERY_PROPERTY
BYTE buffer[4096] = {0};
PSTORAGE_PROPERTY_QUERY query = (PSTORAGE_PROPERTY_QUERY)buffer;
PSTORAGE_PROTOCOL_SPECIFIC_DATA protocolData =
    (PSTORAGE_PROTOCOL_SPECIFIC_DATA)query->AdditionalParameters;

query->PropertyId = StorageDeviceProtocolSpecificProperty;
query->QueryType = PropertyStandardQuery;
protocolData->ProtocolType = ProtocolTypeNvme;
protocolData->DataType = NVMeDataTypeLogPage;
protocolData->ProtocolDataRequestValue = NVME_LOG_PAGE_HEALTH_INFO;
protocolData->ProtocolDataOffset =
sizeof(STORAGE_PROTOCOL_SPECIFIC_DATA);
protocolData->ProtocolDataLength = sizeof(NVME_HEALTH_INFO_LOG);

DeviceIoControl(hDevice, IOCTL_STORAGE_QUERY_PROPERTY,
    buffer, sizeof(buffer), buffer, sizeof(buffer), &bytesReturned,
NULL);

// Температура: ((Temperature[1] << 8 | Temperature[0]) - 273) °C
// PercentageUsed — износ SSD
// AvailableSpare — доступный резерв
```

Интерфейсы и устройства вычислительных машин

3.7.7. Получение температуры

```
// Для NVMe и современных SATA через
StorageDeviceTemperatureProperty
STORAGE_PROPERTY_QUERY tempQuery = {0};
tempQuery.PropertyId = StorageDeviceTemperatureProperty;
tempQuery.QueryType = PropertyStandardQuery;

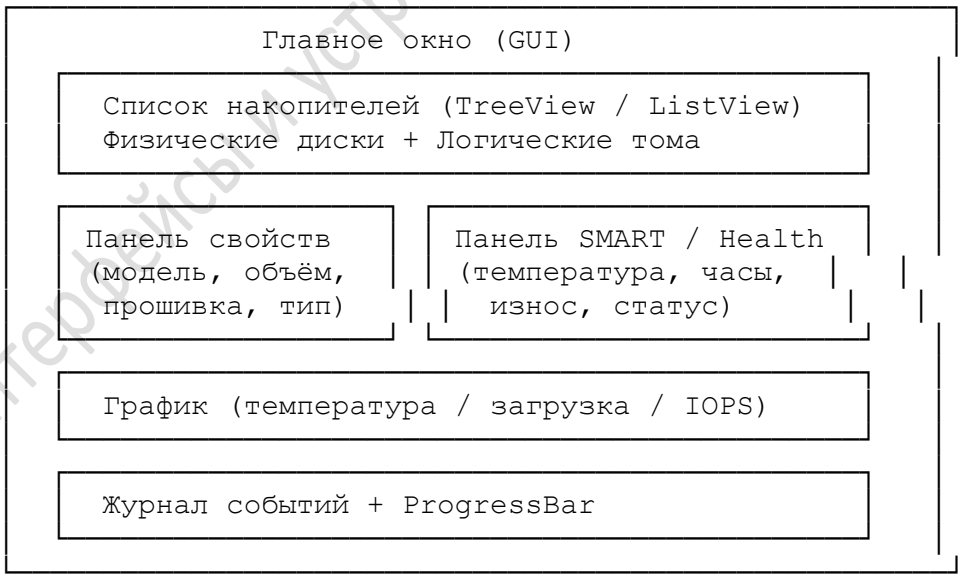
STORAGE_TEMPERATURE_DATA_DESCRIPTOR tempDesc = {0};
DeviceIoControl(hDevice, IOCTL_STORAGE_QUERY_PROPERTY,
    &tempQuery, sizeof(tempQuery),
    &tempDesc, sizeof(tempDesc), &bytesReturned, NULL);
// tempDesc.TemperatureInfo[0].Temperature — температура в °C
```

3.7.8. Мониторинг производительности (IOCTL_DISK_PERFORMANCE)

```
DISK_PERFORMANCE perf;
DeviceIoControl(hDevice, IOCTL_DISK_PERFORMANCE,
    NULL, 0, &perf, sizeof(perf), &bytesReturned, NULL);
// perf.ReadTime, perf.WriteTime — время чтения/записи
// perf.BytesRead, perf.BytesWritten — объём данных
// perf.ReadCount, perf.WriteCount — число операций
```

Примечание: для получения статистики производительности может потребоваться включение счётчиков через diskperf -y.

3.8. Рекомендуемая архитектура приложения (Windows)



3.9. Работа с накопителями в Linux

В Linux информация о накопителях и управление ими доступны через

Интерфейсы устройств вычислительных машин

псевдофайловые системы **sysfs** и **procfs**, а также через специализированные **ioctl**-вызовы. Прямой доступ к ATA-портам возможен через системные вызовы ioperm/iopl (требуется root).

3.9.1. Обнаружение накопителей через sysfs

Ядро Linux экспортирует блочные устройства через /sys/block/ и /sys/class/block/:

- /sys/block/sda/ — SATA-диск;
- /sys/block/nvme0n1/ — NVMe-накопитель;
- /sys/class/nvme/nvme0/ — NVMe-контроллер.

Ключевые файлы:

Путь	Описание
/sys/block/sda/size	Размер в секторах (512 байт)
/sys/block/sda/device/model	Модель устройства
/sys/block/sda/device/vendor	Производитель
/sys/block/sda/device/rev	Ревизия прошивки
/sys/block/sda/queue/physical_block_size	Физический размер сектора
/sys/class/nvme/nvme0/model	Модель NVMe
/sys/class/nvme/nvme0/serial	Серийный номер NVMe
/sys/class/nvme/nvme0/firmware_rev	Версия прошивки NVMe
/sys/class/nvme/nvme0/cntlid	ID контроллера

Для определения типа накопителя (HDD/SSD):

- /sys/block/sda/queue/rotational — 1 для HDD, 0 для SSD;
- /sys/block/sda/queue/discard_granularity — ненулевое значение указывает на поддержку TRIM (SSD).

3.9.2. Получение идентификационной информации (Linux)

Для SATA-накопителей доступна команда **IDENTIFY DEVICE** через ioctl: - **HDIO_GET_IDENTITY** (#include <linux/hdreg.h>) — возвращает структуру hd_driveid (512 байт), совместимую с ATA IDENTIFY.

```
#include <linux/hdreg.h>
#include <sys/ioctl.h>
#include <fcntl.h>

int fd = open("/dev/sda", O_RDONLY);
struct hd_driveid id;
if (ioctl(fd, HDIO_GET_IDENTITY, &id) == 0) {
    // id.model — модель (byte-swapped)
    // id.serial_no — серийный номер
    // id.fw_rev — версия прошивки
    // id.lba_capacity — объём в секторах LBA
```

Интерфейсы и устройства вычислительных машин

```
}
```

Для NVMe-накопителей:

- NVMe Identify Controller через ioctl NVME_IOCTL_ADMIN_CMD (#include <linux/nvme_ioctl.h>);
- структура nvme_admin_cmd, opcode = 0x06 (Identify), nsid = 0, data_len = 4096.

3.9.3. SMART-диагностика в Linux

Для SATA (через SG_IO — SCSI Generic):

```
#include <scsi/sg.h>
#include <sys/ioctl.h>
#include <fcntl.h>

sg_io_hdr_t io_hdr = {0};
unsigned char cdb[16] = {0};
cdb[0] = 0x85; // ATA PASS-THROUGH (16)
cdb[1] = 0x08; // ATA protocol: PIO Data-In
cdb[2] = 0x0E; // T_DIR=1, T_LENGTH=2, BYT_BLOK=1
cdb[3] = 0xD0; // Features: SMART READ DATA
cdb[4] = 0xB0; // Command: SMART
cdb[8] = 0x01; // LBA Mid (0x4F for SMART)
cdb[10] = 0x01; // LBA High (0xC2 for SMART)

unsigned char sense_buffer[32] = {0};
unsigned char data_buffer[512] = {0};

io_hdr.interface_id = 'S';
io_hdr.cmd_len = sizeof(cdb);
io_hdr.mx_sb_len = sizeof(sense_buffer);
io_hdr.dxfer_direction = SG_DXFER_FROM_DEV;
io_hdr.dxfer_len = sizeof(data_buffer);
io_hdr.dxferp = data_buffer;
io_hdr.cmdp = cdb;
io_hdr.sbp = sense_buffer;
io_hdr.timeout = 5000;

ioctl(fd, SG_IO, &io_hdr);
```

Для NVMe:

- NVMe Get Log Page, opcode = 0x02, LID = 0x02 (Health Information Log);
- Структура nvme_admin_cmd с полями opcode, nsid, addr (указатель на буфер), data_len, cdw10 = (0x02 << 16) | (0x7F & (4096/4 - 1)).

Примечание: для работы с SG_IO требуется открыть /dev/sg* или /dev/sd* с правами root (или членство в группе disk).

3.9.4. Температура накопителя (Linux)

- **SATA:** SMART-атрибут 0xC2 (через SG_IO) или утилита hddtemp (порт

Интерфейсы и устройства вычислительных машин

7634);

- **NVMe:** Health Log (температура в Кельвинах, поле Temperature[0..1]);
- **sysfs hwmon:** /sys/class/hwmon/hwmon*/temp1_input (миллиградусы);
- **NVMe sysfs:** /sys/class/nvme/nvme0/hwmon0/temp1_input (если экспортируется драйвером).

3.9.5. Мониторинг производительности (Linux)

Ядро Linux предоставляет статистику ввода-вывода через:
- **/proc/diskstats** — глобальная статистика по всем блочным устройствам;
- **/sys/block/<dev>/stat** — статистика конкретного устройства;
Формат /sys/block/sda/stat (14 полей, ядро 5.5+):

```
reads read_merged reads_sectors read_ms
writes write_merged writes_sectors write_ms
in_progress io_ms weighted_io_ms
discards discard_merged discard_sectors discard_ms
flush_requests flush_ms
```

Из этих данных можно вычислить:
- **IOPS** = (reads_now - reads_prev) / interval;
- **MB/s** = (sectors_now - sectors_prev) * sector_size / interval / 1024 / 1024;
- **Latency** = read_ms / reads (среднее время на операцию).

3.9.6. Анализ использования пространства (Linux)

Для определения свободного/занятого места:
- системный вызов statfs() (или statvfs());
- чтение /proc/mounts для получения списка смонтированных файловых систем.

```
#include <sys/statvfs.h>
struct statvfs buf;
statvfs("/", &buf);
unsigned long long total = buf.f_blocks * buf.f_frsize;
unsigned long long free  = buf.f_bfree  * buf.f_frsize;
```

3.9.7. Прямой доступ к ATA-регистрам через порты ввода-вывода (Linux)

Для низкоуровневого взаимодействия с ATA-контроллером в Linux используются:
- ioperm(0x1F0, 8, 1) — разрешение доступа к регистрам Primary Channel (0x1F0–0x1F7);
- ioperm(0x3F6, 1, 1) — разрешение доступа к регистру управления;
- ioperm(0x170, 8, 1) и ioperm(0x376, 1, 1) — Secondary Channel;
- альтернативно: iopl(3) для доступа ко всем портам.

Интерфейсы и устройства вычислительных машин

Пример программного сброса и чтения регистра состояния:

```
#include <sys/io.h>
#include <unistd.h>

if (iopl(3) == 0) {
    outb(0x04, 0x3F6); // SRST=1 (бит 2)
    usleep(10);
    outb(0x00, 0x3F6); // SRST=0
    usleep(2000);
    unsigned char status = inb(0x1F7); // Чтение SR
    // Ожидание BSY=0, DRDY=1
    iopl(0);
}
```

Важно: на современных системах с AHCI-контроллерами порты 0x1F0/0x170 могут быть не задекларированы BIOS/UEFI (legacy mode отключён). Для выполнения данного варианта рекомендуется использовать виртуальную машину с включённым IDE-режимом (VirtualBox: Storage → Controller → Type = PIIX3/PIIX4).

4. Варианты заданий

ВАРИАНТЫ ДЛЯ WINDOWS

Вариант А1. Базовый монитор накопителей

Реализовать приложение с одним главным окном, содержащим:

- список обнаруженных накопителей (физические диски \\.\PhysicalDriveN с отображением логических томов);
- панель подробной информации о выбранном устройстве: наименование, тип (HDD/SSD), объём, версия прошивки, интерфейс (SATA/NVMe);
- индикатор заполнения памяти (ProgressBar) для каждого логического тома;
- кнопки «Обновить данные» и «Сохранить отчёт»;
- журнал событий с фиксацией операций обновления и ошибок.

Дополнительно: реализовать автообновление данных по таймеру (например, раз в 10 секунд) с возможностью включения/выключения.

Вариант А2. SMART-диагностика

Реализовать приложение, включающее согласованные с преподавателем требования базового задания, а также:

- отдельную вкладку «SMART» с таблицей атрибутов для SATA-накопителей (ID, название, текущее значение, порог, raw-значение, статус);
- отдельную вкладку «NVMe Health» для NVMe-накопителей с отображением: температуры, PercentageUsed, AvailableSpare, PowerCycles,

Интерфейсы и устройства вычислительных машин

PowerOnHours, UnsafeShutdowns;

- индикатор состояния накопителя с цветовой индикацией: зелёный (исправен), жёлтый (предупреждение — например, PercentageUsed > 80% или температура выше 70°C), красный (критическое состояние — AvailableSpare < 10% или критические ошибки);
- сохранение полного SMART-отчёта в файл.

Вариант А3. Анализатор производительности

Реализовать приложение с акцентом на мониторинг производительности:

- получение статистики IOCTL_DISK_PERFORMANCE для выбранного физического диска;
- отображение в реальном времени: IOPS (операций в секунду), MB/s чтения/записи, среднего времени отклика;
- график изменения загрузки диска (процент времени активности) за последние 60 секунд (накопление данных в кольцевом буфере + отрисовка через GDI/GDI+);
- сравнительная панель: одновременный мониторинг двух выбранных дисков (например, HDD vs SSD);
- кнопка «Сбросить статистику».

Вариант А4. Сравнительный анализатор HDD vs SSD

Реализовать приложение, позволяющее сравнивать характеристики накопителей:

- автоматическое определение типа каждого накопителя (HDD/SSD) через StorageDeviceSeekPenaltyProperty, StorageDeviceTrimProperty, StorageDeviceRPMProperty;
- таблица сравнения двух выбранных дисков по параметрам: объём, интерфейс, скорость вращения (для HDD) / TRIM (для SSD), температура, часы работы, износ;
- рекомендации по замене (например, если PercentageUsed > 90% или температура > 75°C — вывод предупреждения);
- экспорт сравнительного отчёта.

Вариант А5. Монитор температуры и предупреждений

Реализовать приложение с акцентом на термический мониторинг:

- непрерывный опрос температуры всех накопителей (SATA через SMART-атрибут 0xC2, NVMe через StorageDeviceTemperatureProperty);
- график изменения температуры каждого накопителя за последние 10 минут;
- настраиваемые пороги температуры (по умолчанию: предупреждение > 55°C, критическая > 70°C);
- всплывающие уведомления (balloon tips / кастомные окна) при

Интерфейсы и устройства вычислительных машин

превышении порогов;

- ведение журнала температурных событий с временными метками.

Вариант А6. Полная диагностическая станция

Реализовать приложение, объединяющее все возможности:

- обнаружение всех накопителей с полной идентификацией (модель, прошивка, серийный номер, интерфейс, размер сектора);
- SMART-диагностика для SATA + NVMe Health для PCIe SSD;
- мониторинг производительности в реальном времени (IOPS, MB/s, latency);
- анализ заполнения дисков с прогнозом (тренд роста занятого места);
- система предупреждений с настраиваемыми порогами (температура, износ, заполнение, reallocations);
- формирование и сохранение комплексного отчёта в формате .txt или .html с таблицами и графиками.

Вариант А7. Низкоуровневый ATA-контроллер через порты ввода-вывода

Реализовать приложение, взаимодействующее с ATA-контроллером напрямую через порты ввода-вывода (без драйверного стека ОС):

- доступ к регистрам Primary Channel (0x1F0–0x1F7) и Secondary Channel (0x170–0x177), а также регистрам управления (0x3F6, 0x376);
- выполнение программного сброса (SRST) через регистр DC;
- ожидание снятия флага BSY и установки DRDY в регистре SR;
- отправка команды IDENTIFY DEVICE (0xEC) и чтение 512 байт ответа в режиме PIO (чтение 256 слов из регистра данных 0x1F0);
- парсинг данных IDENTIFY DEVICE: модель (word 27–46), серийный номер (word 10–19), версия прошивки (word 23–26), объём (word 60–61 LBA28 или 100–103 LBA48), признак LBA, Nominal Media Rotation Rate (word 217: 0x0001 = SSD);
- определение «пустого» канала: если регистр состояния возвращает 0x00 или 0xFF — устройство отсутствует;
- отображение полученной информации в GUI и сохранение отчёта.

Требования к окружению: для выполнения данного варианта требуется:

- запуск от имени администратора;
- 32-битная Windows (x86) или использование драйвера разрешения портов (GiveIO, WinIo) в 64-битной системе;
- на x64 без драйвера — работа внутри виртуальной машины с проброской портов.

Вариант А8. Диагностика и тестирование ATA-канала

Реализовать приложение для диагностики и тестирования ATA-каналов

Интерфейсы и устройства вычислительных машин

через порты ввода-вывода:

- сканирование Primary и Secondary каналов, определение наличия Master/Slave;
- определение типа устройства (ATA vs ATAPI) через сигнатуры в регистре CL/CH после сброса;
- чтение одного сектора в режиме CHS (команда 0x20 — READ SECTORS) и в режиме LBA (команда 0x20 или 0x24 — READ SECTORS EXT);
- измерение времени чтения сектора в режиме PIO (latency) и отображение результатов;
- выполнение команды SET FEATURES (0xEF) для переключения PIO Mode (подкоманда 0x03) и включения/выключения Ultra DMA (подкоманда 0x03 с feature 0x05–0x07);
- формирование отчёта с картой каналов, идентификацией устройств и результатами тестов производительности.

Важно: все операции записи (Set Features) выполнять только в виртуальной машине!

ВАРИАНТЫ ДЛЯ LINUX

Общее для Linux-вариантов: рекомендуется использование ioctl. Для вариантов Б7–Б8 (порты) требуется iopl(3) или ioperm с правами root. Запрещается использование высокоуровневых обёрток (libatasmart, nvme-cli, hdparm как библиотека) без согласования с преподавателем.

Вариант Б1. Базовый монитор накопителей (Linux)

Реализовать приложение с одним главным окном, содержащим:

- список обнаруженных накопителей (чтение /sys/block/, /sys/class/nvme/, отображение логических томов через statvfs или /proc/mounts);
- панель подробной информации: наименование (/sys/.../model), тип (HDD/SSD через rotational), объём (/sys/.../size), версия прошивки (/sys/class/nvme/.../firmware_rev или HDIO_GET_IDENTITY), интерфейс (SATA/NVMe);
- индикатор заполнения памяти (ProgressBar) для каждого смонтированного раздела (statvfs);
- кнопки «Обновить данные» и «Сохранить отчёт»;
- журнал событий с фиксацией операций обновления и ошибок.

Дополнительно: реализовать автообновление по таймеру (например, раз в 10 секунд).

Вариант Б2. SMART-диагностика (Linux)

Реализовать приложение, включающее все требования базового задания, а также:

- отдельную вкладку «SMART» с таблицей атрибутов для SATA-

Интерфейсы устройства вычислительных машин

накопителей, полученных через SG_IO (ATA PASS THROUGH) или HDIO_DRIVE_TASK;

- отдельную вкладку «NVMe Health» с отображением: температуры, PercentageUsed, AvailableSpare, PowerCycles, PowerOnHours (через NVME_IOCTL_ADMIN_CMD с Log Page 0x02);
- индикатор состояния накопителя с цветовой индикацией (зелёный/жёлтый/красный);
- сохранение полного SMART-отчёта в файл.

Вариант Б3. Анализатор производительности (Linux)

Реализовать приложение с акцентом на мониторинг производительности:

- чтение /sys/block/<dev>/stat или /proc/diskstats для выбранного диска;
- отображение в реальном времени: IOPS, MB/s чтения/записи, среднего времени отклика (расчёт по полям read_ms / reads);
- график изменения загрузки диска за последние 60 секунд (накопление в кольцевом буфере + отрисовка через Cairo / Qt Charts);
- сравнительная панель: одновременный мониторинг двух выбранных дисков;
- кнопка «Сбросить статистику».

Вариант Б4. Сравнительный анализатор HDD vs SSD (Linux)

Реализовать приложение, позволяющее сравнивать характеристики накопителей:

- автоматическое определение типа (HDD/SSD) через /sys/block/.../queue/rotational и discard_granularity;
- таблица сравнения двух дисков по параметрам: объём, интерфейс, TRIM, температура (если доступна), часы работы, износ (NVMe PercentageUsed);
- рекомендации по замене;
- экспорт сравнительного отчёта.

Вариант Б5. Монитор температуры и предупреждений (Linux)

Реализовать приложение с акцентом на термический мониторинг:

- непрерывный опрос температуры (SATA через SMART-атрибут 0xC2 / SG_IO, NVMe через Health Log, или /sys/class/hwmon/);
- график изменения температуры за последние 10 минут;
- настраиваемые пороги температуры (предупреждение > 55°C, критическая > 70°C);
- всплывающие уведомления (libnotify через D-Bus или кастомные окна) при превышении порогов;
- ведение журнала температурных событий.

Интерфейсы устройства вычислительных машин

Вариант Б6. Полная диагностическая станция (Linux)

Реализовать приложение, объединяющее все возможности:

- обнаружение всех накопителей через sysfs с полной идентификацией;
- SMART-диагностика для SATA (SG_IO) + NVMe Health (ioctl);
- мониторинг производительности в реальном времени (/proc/diskstats);
- анализ заполнения дисков (statvfs) с прогнозом;
- система предупреждений с настраиваемыми порогами;
- формирование и сохранение комплексного отчёта в .txt или .html.

Вариант Б7. Низкоуровневый ATA-контроллер через порты ввода-вывода (Linux)

Реализовать приложение, взаимодействующее с ATA-контроллером напрямую через порты ввода-вывода:

- доступ к регистрам Primary Channel (0x1F0–0x1F7, 0x3F6) и Secondary Channel (0x170–0x177, 0x376) через ioperm/iopl;
- выполнение программного сброса (SRST) через регистр DC (0x3F6);
- ожидание снятия BSY и установки DRDY;
- отправка команды IDENTIFY DEVICE (0xEC) и чтение 512 байт ответа в режиме PIO (чтение 256 слов из 0x1F0 функциями inw);
- парсинг данных IDENTIFY DEVICE: модель, серийный номер, прошивка, объём, признак LBA, Nominal Media Rotation Rate;
- определение «пустого» канала (0x00 или 0xFF в регистре состояния);
- отображение информации в GUI и сохранение отчёта.

Требования: запуск от root, iopl(3). Рекомендуется виртуальная машина с IDE-контроллером (PIIX3).

Вариант Б8. Диагностика и тестирование ATA-канала (Linux)

Реализовать приложение для диагностики и тестирования ATA-каналов через порты:

- сканирование Primary/Secondary каналов, определение Master/Slave;
- определение ATA vs ATAPI через сигнатуры CL/CH после сброса;
- чтение сектора в режиме CHS (0x20) и LBA (0x20 / 0x24) через порты;
- измерение latency PIO (время чтения одного сектора);
- выполнение SET FEATURES (0xEF) для переключения PIO Mode;
- формирование отчёта с картой каналов и тестами.

Важно: операции записи (Set Features) — только в VM!

5. Контрольные вопросы

1. Опишите эволюцию интерфейсов хранения данных: в чём принципиальные отличия PATA, SATA и NVMe?
2. Какие компоненты определяет спецификация ATA? Чем ведущее

Интерфейсы и устройства вычислительных машин

устройство отличается от ведомого?

3. Перечислите регистры ATA-устройства и их назначение. Как формируется адрес сектора в режиме LBA?

4. Опишите назначение битов регистра состояния SR (BSY, DRDY, DRQ, ERR, DF).

5. В чём разница между протоколами обмена PIO, DMA и Ultra DMA? Какие преимущества даёт каждый из них?

6. Что такое AHCI? Какие возможности он добавляет по сравнению с legacy IDE-режимом?

7. Что такое NVMe и почему он обеспечивает более высокую производительность, чем SATA/AHCI?

8. Что такое SMART? Какие ключевые атрибуты SMART используются для оценки состояния накопителя?

9. Как программно определить тип накопителя (HDD или SSD) в Windows без использования WMI?

10. Какие IOCTL-коды используются для получения информации о накопителе в Windows? Опишите назначение каждого.

11. Как получить SMART-данные от SATA-накопителя через IOCTL_ATA_PASS_THROUGH? Опишите последовательность действий.

12. Как получить информацию о здоровье NVMe-накопителя (температуру, износ, доступный резерв)? Какие структуры данных используются?

13. Как получить температуру накопителя через IOCTL_STORAGE_QUERY_PROPERTY? Для каких типов устройств доступен этот метод?

14. Как осуществляется прямой доступ к ATA-регистрам через порты ввода-вывода в Windows? Какие ограничения существуют на x64 системах?

15. Опишите полный протокол выполнения команды Identify Device через порты ввода-вывода (все шаги от ожидания BSY до чтения данных).

16. Как определить наличие устройства на ATA-канале через порты ввода-вывода? Какое значение возвращает регистр состояния при отсутствии устройства?

17. Как организовать мониторинг состояния накопителя в реальном времени без блокировки GUI? Опишите подходы с использованием таймеров и отдельных потоков.

18. Как в Linux получить список блочных устройств и их характеристики через sysfs и udev? Опишите назначение файлов /sys/block/.../device/model, /sys/block/.../queue/rotational.

19. Какие ioctl-коды используются в Linux для получения SMART-данных SATA-накопителя (SG_IO, HDIO_GET_IDENTITY) и информации о здоровье NVMe (NVME_IOCTL_ADMIN_CMD)? Опишите структуры данных.

20. Как организовать прямой доступ к ATA-регистрам через порты ввода-

Интерфейсы и устройства вычислительных машин

вывода в Linux? Какие системные вызовы (ioperm, iopl) требуются и какие ограничения существуют?

21. Как получить статистику производительности диска в Linux через /proc/diskstats и /sys/block/.../stat? Какие поля содержат IOPS и пропускную способность?

22. Какие инструменты Linux (hdparm, smartctl, nvme-cli) можно использовать для диагностики накопителей, и какие задачи они решают? Почему их прямое использование в коде в рамках данной работы ограничено?

6. Список рекомендуемой литературы

6.1 Основная литература

1. Таненбаум Э. **Архитектура компьютера**. — 6-е изд. — СПб.: Питер, 2013. — Глава 5 (организация ввода-вывода, дисковые устройства).
2. Microsoft Learn. Working with NVMe Drives — официальная документация по работе с NVMe-накопителями в Windows (IOCTL, структуры данных).
3. Microsoft Learn. IOCTL_STORAGE_QUERY_PROPERTY — справочник по IOCTL для запроса свойств устройств хранения.
4. Microsoft Learn. IOCTL_ATA_PASS_THROUGH — документация по ATA pass-through для отправки ATA-команд.

6.2 Дополнительная литература (Windows)

5. Wikipedia. ATA/ATAPI — справочная статья по интерфейсу ATA.
6. Wikipedia. NVMe — справочная статья по протоколу NVMe.
7. Wikipedia. S.M.A.R.T. — описание технологии SMART и таблица атрибутов.
8. Stack Overflow. Detecting SSD in Windows - C++ — пример определения типа накопителя (HDD/SSD) через IOCTL.
9. Stack Overflow. NVMe S.M.A.R.T. data retrieval — пример чтения NVMe Health Log через IOCTL_STORAGE_QUERY_PROPERTY.
10. smartmontools. os_win32.cpp Source — исходный код кроссплатформенной утилиты SMART для Windows (справочный материал).
11. Рихтер Дж., Назаре К. **Windows via C/C++. Программирование на платформе Microsoft Windows**. — 5-е изд. — СПб.: Символ-Плюс, 2012. — Главы 7, 10 (работа с устройствами, асинхронный ввод-вывод).
12. Дунаев В. **Взаимодействие приложений с устройствами в ОС Windows**. — М.: ДМК Пресс, 2010. — Разделы 3.1–3.3 (драйверный стек, IOCTL).

Интерфейсы устройств вычислительных машин

6.3 Дополнительная литература (Linux)

13. Linux Kernel Documentation. SCSI Generic (sg) driver — документация по интерфейсу SG_IO для ATA PASS THROUGH.

14. Linux Kernel Documentation. NVMe driver — документация по NVMe-интерфейсу ядра, ioctl-кодам.

15. man pages: hdio(2), sg_io(3), iopl(2), ioperm(2), statvfs(2), inotify(7), udev(7).

16. The Linux Programming Interface. Michael Kerrisk. — No Starch Press, 2010. — Chapters 5 (Files), 14 (File Systems), 23 (Timers).

17. NVM Express Base Specification — официальная спецификация NVMe (NVMe Health Log, Identify Controller).

18. Linux Block I/O Tracing — документация по /sys/block/.../stat и /proc/diskstats.

6.4 Справочные ресурсы

19. PCI ID Repository — база данных идентификаторов PCI-устройств (для определения контроллеров хранения).

20. Microsoft Docs. STORAGE_TEMPERATURE_INFO structure — структура данных для получения температуры накопителя.

21. ATA/ATAPI Command Set specifications — спецификации T13 (ATA-8 и новее).

22. OSDev Wiki. ATA PIO Mode — справочник по программированию ATA-контроллера в режиме PIO.

Интерфейсы и устройства вычислительных машин